

Presentation On

Offline RL: Conservative Q-Learning

Nirjhar, Chinmay, Maulik

March 25, 2026

Chennai Mathematical Institute

Plan

Offline Reinforcement Learning

Behavior Cloning

Offline Q-Learning

Conservative Q-Learning

Dataset Collection & Online DQN

Experiments

Offline Reinforcement Learning

Offline RL: Setting and Goal

Offline RL: learn a policy π from a *fixed dataset*

$$\mathcal{D} = \{(s_i, a_i, r_i, s'_i)\}_{i=1}^N$$

collected by some behavior policy π_β , with *no further environment interaction*.

MDP:

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, T, r, \gamma), \quad J(\pi) = \mathbb{E}_{\tau \sim p_\pi(\tau)} \left[\sum_{t=0}^H \gamma^t r(s_t, a_t) \right].$$

Difference from online / off-policy RL

- Online RL keeps collecting fresh data.
- Standard off-policy RL reuses replay-buffer data *and* still interacts.
- Offline RL trains only on $\mathcal{D}$; deployment happens after training.

Main promise: use large logged datasets to learn decision-making policies in settings where interaction is expensive, risky, or impossible.

Formal Preliminaries

For a policy $\pi(a|s)$, define

$$V^\pi(s) = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \mid s_0 = s, \pi \right],$$

$$Q^\pi(s, a) = r(s, a) + \gamma \mathbb{E}_{s' \sim T(\cdot|s, a), a' \sim \pi(\cdot|s')} [Q^\pi(s', a')].$$

Bellman operators

$$(\mathcal{B}^\pi Q)(s, a) = r(s, a) + \gamma \mathbb{E}_{s', a'} [Q(s', a')],$$

$$(\mathcal{B}^* Q)(s, a) = r(s, a) + \gamma \mathbb{E}_{s' \sim T(\cdot|s, a)} \left[\max_{a'} Q(s', a') \right].$$

Offline methods typically reuse these same operators, but only through samples from the static dataset $\mathcal{D}$.

What Makes Offline RL Difficult? (1/2)

Key Challenge: No Exploration

- Learning is restricted to a fixed dataset $\mathcal{D}$
- No interaction with environment $\Rightarrow$ no new data collection
- If high-reward regions are not in $\mathcal{D}$, they cannot be discovered

Assumption

- Dataset $\mathcal{D}$ has sufficient coverage of high-reward transitions

More Subtle Challenge: Counterfactual Reasoning

- Offline RL must answer: *“What if we take unseen actions?”*
- Learned policy must differ from behavior policy to improve
- Violates standard i.i.d. assumption in supervised learning

Distributional Shift (2/2)

Core Problem: Distributional Shift

- Training distribution: $s \sim d_{\pi_\beta}(s)$
- Evaluation distribution: $s \sim d_\pi(s)$
- These can be very different

Why does this happen?

- New policy visits different states
- Maximizing return pushes toward unseen actions

Mitigation Strategy

- Constrain learned policy:

$$\pi(a|s) \approx \pi_\beta(a|s)$$

- Limits state distribution shift

Takeaway

- Offline RL = learning under distribution mismatch

Behavior Cloning

Behavior Cloning (BC)

Idea: ignore long-horizon credit assignment and directly imitate the dataset actions.

For discrete actions (CartPole):

$$\pi_{\theta} = \arg \max_{\pi} \mathbb{E}_{(s,a) \sim \mathcal{D}} [\log \pi(a|s)].$$

For continuous actions (HalfCheetah):

$$\pi_{\theta} = \arg \min_{\pi} \mathbb{E}_{(s,a) \sim \mathcal{D}} [\|\pi(s) - a\|^2]$$

Advantages

- Simple and stable
- Pure supervised learning
- Strong baseline when data is high-quality

Limitations

- Cannot improve beyond data quality
- No explicit reward optimization
- Suffers from covariate shift across time

When Does BC Work Well?

BC works well when

- behavior policy is already near-optimal,
- dataset covers the states visited at deployment,
- task is short-horizon or robust to action noise,
- reward is hard to use but expert actions are available.

BC is usually a lower-variance method

But its bias is strong:

$$\pi_{\text{BC}} \approx \pi_{\beta}.$$

So if π_{β} is poor, BC is capped.

This makes BC a crucial baseline in offline RL:

Can our RL method beat imitation, without going OOD?

Offline Q-Learning

From Dynamic Programming to Offline Q-Learning

In standard offline Q-learning, we regress Q to Bellman targets built from the dataset:

$$Q_{k+1} = \arg \min_Q \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[\left(Q(s,a) - \underbrace{(r + \gamma \max_{a'} Q_k(s', a'))}_{\text{target}} \right)^2 \right].$$

Policy extraction:

$$\pi_Q(s) = \arg \max_a Q(s, a).$$

Offline version: same Bellman regression as usual, but the replay buffer is fixed once and for all.

This is attractive because Q-learning can, in principle, improve beyond the behavior policy.

Why Naive Offline Q-Learning Fails?

The failure mechanism

$$y = r + \gamma \max_{a'} Q_k(s', a').$$

But the maximizing action a' may be far outside the support of $\mathcal{D}$ at state s' .

Then:

1. $Q_k(s', a')$ is estimated by extrapolation.
2. Random approximation errors can make it spuriously large.
3. The max operator selects precisely those bad overestimates.
4. The Bellman backup propagates them backward.

Key Insight

Offline Q-learning fails due to overestimation on OOD actions.

Offline Q-Learning: Mathematical View of the Problem

Actor-critic style offline RL performs

$$\hat{Q}_{k+1}^{\pi} \leftarrow \arg \min_Q \mathbb{E}_{(s,a,s') \sim \mathcal{D}} \left[\left(Q(s,a) - [r + \gamma \mathbb{E}_{a' \sim \pi_k(\cdot|s')} \hat{Q}_k^{\pi}(s',a')] \right)^2 \right],$$

followed by policy improvement

$$\pi_{k+1} \leftarrow \arg \max_{\pi} \mathbb{E}_{s \sim \mathcal{D}, a \sim \pi(\cdot|s)} [\hat{Q}_{k+1}^{\pi}(s,a)].$$

Mismatch:

Q is trained on $a \sim \pi_{\beta}(\cdot|s)$, but backed up / optimized on $a \sim \pi_k(\cdot|s)$.

This is the central *action distribution shift* problem in offline dynamic programming.

OOD Experiments

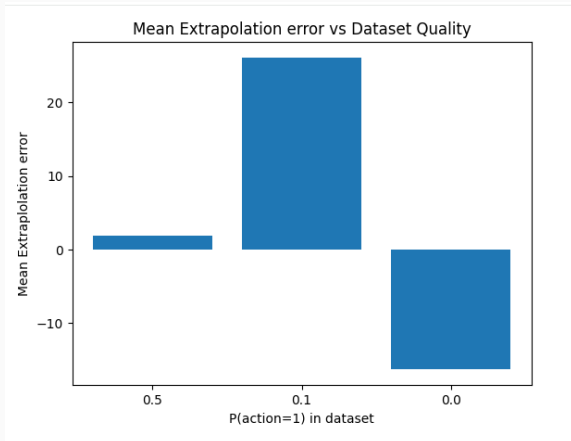


Figure shows mean extrapolation error for different naive learner derived datasets with different coverage of actions (to simulate OOD actions)

Classical Remedies Before CQL

Two broad families appear in the offline RL literature:

1. Policy-constraint methods

Keep the learned policy close to the dataset policy:

$$D(\pi(\cdot|s), \pi_\beta(\cdot|s)) \leq \varepsilon$$

using KL, MMD, Wasserstein, support constraints, etc.

2. Uncertainty / pessimism methods

Estimate uncertainty on unsupported actions and reduce their values:

$$Q_{\text{pess}}(s, a) \approx \mathbb{E}[Q(s, a)] - \alpha \cdot \text{Unc}(s, a).$$

Tradeoff

- Constraints reduce OOD actions but may over-restrict improvement.
- Pessimism avoids behavior-model estimation, but too much pessimism can under-exploit useful actions.

Conservative Q-Learning

Conservative Q-Learning (CQL): Main Idea

Instead of only constraining the policy, **regularize the Q-function itself** so that unsupported actions receive *smaller* values.

Desired property:

$$\mathbb{E}_{a \sim \pi(\cdot|s)}[\hat{Q}(s, a)] \leq V^\pi(s).$$

So the learned Q-function is a **lower bound** on policy value, preventing optimistic exploitation of OOD actions.

Why this is appealing

- Works directly at the Bellman backup level
- No need to explicitly estimate π_β in the basic practical variant
- Applies to both actor-critic and Q-learning style methods

Conservative Off-Policy Evaluation

We observe an offline dataset $\mathcal{D}$ collected by a behavior policy $\pi_\beta(a \mid s)$, and want to evaluate a target policy π .

Goal: estimate $V^\pi(s)$ without overestimating unsupported actions.

CQL adds a conservative penalty to the usual Bellman objective.

Since offline Q-learning mainly extrapolates over *actions* at observed states, we choose

$$\mu(s, a) = d^{\pi_\beta}(s) \mu(a \mid s).$$

So the state marginal stays on the dataset, while conservatism is applied through the action distribution.

Basic Conservative Critic Update

$$\hat{Q}^{k+1} = \arg \min_Q \alpha \mathbb{E}_{s \sim \mathcal{D}, a \sim \mu(\cdot|s)}[Q(s, a)] + \frac{1}{2} \mathbb{E}_{(s, a, s') \sim \mathcal{D}} \left[(Q(s, a) - \hat{\mathcal{B}}^\pi \hat{Q}^k(s, a))^2 \right].$$

Interpretation

- Bellman term fits the critic to the target policy.
- Conservative term pushes Q-values downward under μ .

Theorem. for sufficiently large α ,

$$\hat{Q}^\pi(s, a) \leq Q^\pi(s, a),$$

so this update gives a pointwise lower bound.

Proof (1/4)

Start with

$$J(Q) = \alpha \mathbb{E}_{s \sim \mathcal{D}, a \sim \mu(\cdot | s)}[Q(s, a)] + \frac{1}{2} \mathbb{E}_{(s, a, s') \sim \mathcal{D}} \left[(Q(s, a) - \hat{B}^\pi \hat{Q}^k(s, a))^2 \right].$$

Using empirical state-action frequencies:

$$\begin{aligned} J(Q) &= \alpha \sum_s d^{\pi_\beta}(s) \sum_a \mu(a | s) Q(s, a) \\ &\quad + \frac{1}{2} \sum_s d^{\pi_\beta}(s) \sum_a \hat{\pi}_\beta(a | s) \left(Q(s, a) - \hat{B}^\pi \hat{Q}^k(s, a) \right)^2. \end{aligned}$$

Fix (s, a) . Then

$$\begin{aligned} \frac{\partial J}{\partial Q(s, a)} &= \alpha d^{\pi_\beta}(s) \mu(a | s) \\ &\quad + d^{\pi_\beta}(s) \hat{\pi}_\beta(a | s) \left(Q(s, a) - \hat{B}^\pi \hat{Q}^k(s, a) \right). \end{aligned}$$

Setting the derivative to zero gives the tabular update:

$$\hat{Q}^{k+1}(s, a) = \hat{\mathcal{B}}^\pi \hat{Q}^k(s, a) - \alpha \frac{\mu(a | s)}{\hat{\pi}_\beta(a | s)}$$

Thus, at every iteration,

$$\hat{Q}^{k+1}(s, a) \leq \hat{\mathcal{B}}^\pi \hat{Q}^k(s, a).$$

This is the one-step conservative correction.

Proof (3/4)

The empirical Bellman operator differs from the true one:

$$\left| \hat{\mathcal{B}}^\pi Q(s, a) - \mathcal{B}^\pi Q(s, a) \right| \leq \frac{C_{r, T, \delta} R_{\max}}{(1 - \gamma) \sqrt{|D(s, a)|}}.$$

So,

$$\hat{Q}^{k+1}(s, a) \leq \mathcal{B}^\pi \hat{Q}^k(s, a) - \alpha \frac{\mu(a | s)}{\hat{\pi}_\beta(a | s)} + \frac{C_{r, T, \delta} R_{\max}}{(1 - \gamma) \sqrt{|D(s, a)|}}.$$

Now let $\hat{Q}^\pi := \lim_{k \rightarrow \infty} \hat{Q}^k$. At the fixed point,

$$\begin{aligned} \hat{Q}^\pi &\leq \mathcal{B}^\pi \hat{Q}^\pi - \alpha \frac{\mu}{\hat{\pi}_\beta} + \frac{C_{r, T, \delta} R_{\max}}{(1 - \gamma) \sqrt{|D|}} \\ &= R + \gamma P^\pi \hat{Q}^\pi - \alpha \frac{\mu}{\hat{\pi}_\beta} + \frac{C_{r, T, \delta} R_{\max}}{(1 - \gamma) \sqrt{|D|}}. \end{aligned}$$

Proof (4/4): Final Bound

$$(I - \gamma P^\pi) \hat{Q}^\pi \leq R - \alpha \frac{\mu}{\hat{\pi}_\beta} + \frac{C_{r,T,\delta} R_{\max}}{(1 - \gamma) \sqrt{|D|}}.$$

Applying $(I - \gamma P^\pi)^{-1}$,

$$\hat{Q}^\pi \leq (I - \gamma P^\pi)^{-1} R - \alpha (I - \gamma P^\pi)^{-1} \frac{\mu}{\hat{\pi}_\beta} + (I - \gamma P^\pi)^{-1} \frac{C_{r,T,\delta} R_{\max}}{(1 - \gamma) \sqrt{|D|}}.$$

Since $Q^\pi = (I - \gamma P^\pi)^{-1} R$, we obtain

$$\boxed{\hat{Q}^\pi \leq Q^\pi - \alpha (I - \gamma P^\pi)^{-1} \frac{\mu}{\hat{\pi}_\beta} + (I - \gamma P^\pi)^{-1} \frac{C_{r,T,\delta} R_{\max}}{(1 - \gamma) \sqrt{|D|}}}$$

Therefore, if α is large enough, then

$$\hat{Q}^\pi(s, a) \leq Q^\pi(s, a) \quad \forall (s, a).$$

If $\hat{\mathcal{B}}^\pi = \mathcal{B}^\pi$, the sampling term vanishes, so any $\alpha > 0$ suffices.

CQL Objective: A Tighter Lower Bound

The previous update can be too pessimistic. CQL therefore preserves values on data-supported actions:

$$\hat{Q}^{k+1} = \arg \min_Q \alpha \left(\mathbb{E}_{s \sim \mathcal{D}, a \sim \mu(\cdot|s)}[Q(s, a)] - \mathbb{E}_{s \sim \mathcal{D}, a \sim \hat{\pi}_\beta(\cdot|s)}[Q(s, a)] \right) + \frac{1}{2} \mathbb{E}_{(s, a, s') \sim \mathcal{D}} \left[(Q(s, a) - \hat{\mathcal{B}}^\pi \hat{Q}^k(s, a))^2 \right].$$

- penalize broad actions via μ
- preserve in-data actions via $\hat{\pi}_\beta$

Theorem. If $\mu = \pi$, then with suitable α ,

$$\mathbb{E}_{a \sim \pi(\cdot|s)}[\hat{Q}^\pi(s, a)] \leq V^\pi(s).$$

If $\mu = \pi$, then $\forall s \in \mathcal{D}$,

$$\begin{aligned}\hat{V}^\pi(s) \leq V^\pi(s) - \alpha & \left[(I - \gamma P^\pi)^{-1} \mathbb{E}_\pi \left[\frac{\pi}{\hat{\pi}_\beta} - 1 \right] \right] (s) \\ & + \left[(I - \gamma P^\pi)^{-1} \frac{C_{r,T,\delta} R_{\max}}{(1-\gamma)\sqrt{|D|}} \right] (s).\end{aligned}$$

Conservative Q-Learning for Offline RL: CQL($\mathcal{R}$)

We now move from **conservative evaluation** to **offline policy learning**.

- Solving above equation with $\mu = \pi$ gives a lower bound on the value of a fixed policy π .
- However, repeating full off-policy evaluation for every policy iterate $\hat{\pi}_k$ is expensive.

Instead, choose $\mu(a | s)$ to approximate the policy that would maximize the current critic. This gives the general CQL family:

$$\min_Q \max_{\mu} \alpha \left(\mathbb{E}_{s \sim \mathcal{D}, a \sim \mu(\cdot | s)} [Q(s, a)] - \mathbb{E}_{s \sim \mathcal{D}, a \sim \hat{\pi}_\beta(\cdot | s)} [Q(s, a)] \right) \\ + \frac{1}{2} \mathbb{E}_{(s, a, s') \sim \mathcal{D}} \left[(Q(s, a) - \hat{\mathcal{B}}^{\pi_k} \hat{Q}^k(s, a))^2 \right] + R(\mu).$$

Interpretation

- $\max_{\mu}$: finds actions on which the critic is overly optimistic,
- $\min_Q$: suppresses those unsupported high values,
- $R(\mu)$: regularizes the adversarial distribution μ .

Variant: CQL($\mathcal{H}$)

A practical version chooses a KL-style regularizer, yielding the **log-sum-exp** form:

$$\min_Q \alpha \mathbb{E}_{s \sim \mathcal{D}} \left[\log \sum_a \exp(Q(s, a)) - \mathbb{E}_{a \sim \hat{\pi}_\beta(\cdot|s)}[Q(s, a)] \right] \\ + \frac{1}{2} \mathbb{E}_{(s, a, s') \sim \mathcal{D}} \left[(Q(s, a) - \hat{B}^{\pi_k} \hat{Q}^k(s, a))^2 \right].$$

Heuristic reading

- $\log \sum_a e^{Q(s,a)}$ pushes down all potentially high actions.
- The data expectation term protects Q-values on actions actually seen in the dataset.
- Net effect: *increase the gap* between in-dataset and unsupported actions.

Key Theoretical Guarantees of CQL

Theorem (Lower-bound policy value)

With suitable α , CQL learns a Q -function such that

$$\hat{V}^{\pi}(s) = \mathbb{E}_{a \sim \pi(\cdot|s)}[\hat{Q}^{\pi}(s, a)] \leq V^{\pi}(s)$$

for dataset states s .

Theorem (Gap expansion)

CQL increases the difference between values of in-distribution actions and overestimated OOD actions.

Conceptual summary

Standard offline Q-learning is often too optimistic; CQL deliberately becomes pessimistic in exactly the directions where offline extrapolation is unreliable.

Algorithm 1: Conservative Q-Learning

Algorithm 1 CQL (Q-learning or actor-critic)

1: Initialize critic Q_θ and optionally actor π_ϕ

2: **for** $t = 1, \dots, N$ **do**

3: Update critic with CQL loss:

$$\theta_t = \theta_{t-1} - \eta_Q \nabla_\theta \mathcal{L}_{\text{CQL}}(\theta)$$

$\mathcal{B}^*$ for Q-learning, $\mathcal{B}^{\pi_{\phi_t}}$ for actor-critic

4: **if** actor-critic **then**

5: Update actor with SAC-style objective:

$$\phi_t = \phi_{t-1} + \eta_\pi \mathbb{E}_{s \sim \mathcal{D}, a \sim \pi_\phi} [Q_\theta(s, a) - \log \pi_\phi(a|s)]$$

6: **end if**

7: **end for**

CQL vs BC vs Naive Offline Q-Learning

Method	Can improve beyond data?	Main risk	Typical behavior
Behavior Cloning	No / very limited	Covariate shift, imitation ceiling	Stable, strong with expert data
Naive Offline Q-learning	Yes, in principle	OOD overestimation, extrapolation error	Often unstable or over-optimistic
CQL	Yes	Too much pessimism if α is very large	Safer improvement through lower-bound values

Dataset Collection & Online DQN

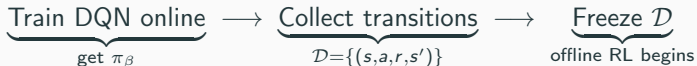
Overview: From Online Training to Offline Dataset

Key idea: We first train a behavior policy π_β online, then *freeze* it and use it to collect a static dataset $\mathcal{D}$ for offline training.

Why train online first?

- A random policy produces only low-quality data
- We need a trained policy to generate the *expert* and *medium* datasets
- The quality of π_β directly determines the ceiling of offline methods

Pipeline



Online DQN: Algorithm

Double DQN with step-based epsilon decay

Q-function update (Bellman regression):

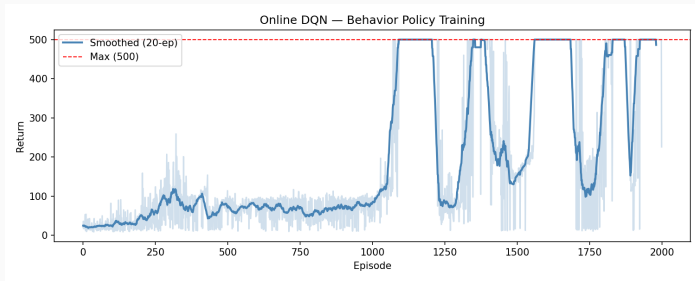
$$Q_{k+1} = \arg \min_Q \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}_{\text{online}}} \left[\left(Q(s,a) - r + \underbrace{\gamma Q_{\text{target}}(s', \arg \max_{a'} Q(s', a'))}_{\text{Double DQN target}} \right)^2 \right]$$

Key implementation choices:

- Double DQN target - reduces overestimation
- Step-based ϵ -decay - more accurate than episode-based
- ϵ decays from 1.0 $\rightarrow$ 0.05 over 20,000 steps
- Target network updated every 500 steps
- Learning rate schedule: $\times 0.5$ at episodes 500, 1000, 1500

Best weights tracking: we save the policy checkpoint with the highest 100-episode average return.

Online DQN: Training Curve



Phase	Episodes	Behavior
Exploration	0 - 1000	ϵ high, mostly random, slow learning
Breakthrough	~ 1100	Policy suddenly converges, return spikes to 500
Oscillation	1100 - 2000	DQN instability - Q-values overwritten repeatedly
Best avg return		500.0 (saved by best-weights tracker)

The oscillation is the *unlearning effect*, even in the online setting. In offline RL, there is no recovery mechanism.

Hyperparameters: Dataset Collection

Parameter	Value	Why
Episodes per dataset	200	Enough transitions for stable training
Max steps per episode	500	CartPole-v1 maximum
ϵ random	1.0	Pure random — no policy used
ϵ medium	0.3	70% policy, 30% random
ϵ expert	0.12	Near-greedy, small noise for diversity
Buffer capacity	200,000	Fits all transitions comfortably
Random seed	1	Fixed for reproducibility

Dataset Collection Strategy

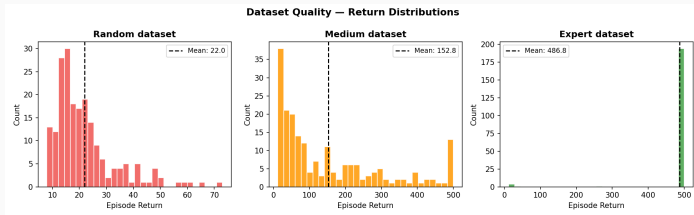
Three datasets collected from π_β with varying levels of randomness:

Dataset	Policy	ϵ	Interpretation
Random	None	1.0	Pure exploration, no learned behavior
Medium	π_β	0.3	Policy acts 70% of the time
Expert	π_β	0.05	Near-greedy, slight randomness for diversity

Why $\epsilon = 0.05$ for expert, not 0?

- $\epsilon = 0$ gives a perfectly narrow dataset - every episode identical
- Narrow data causes severe distribution shift for offline methods (Levine et al. 2020)
- $\epsilon = 0.05$ introduces some diversity while keeping quality high

Dataset Statistics



Dataset	Mean Return	Std Return	Transitions
Random	22.0	11.5	4,394
Medium	152.8	147.1	30,562
Expert	486.8	76.2	97,366

The medium dataset shows a right-skewed distribution, most episodes fail early (~ 10 – 50 return) but a long tail reaches up to 500, reflecting the 30% random action rate. The expert dataset is heavily concentrated near 500 ($\text{std} = 76.2$), with only rare failures due to the 12% noise rate.

Experiments

Experimental Setup

Environment

- CartPole-v1

Method

- Behavior Cloning (BC)

Offline datasets

- Random dataset
- Expert dataset
- Medium dataset

Metrics

- Mean evaluation return
- Training loss (cross-entropy for BC)
- State distribution mismatch
(distribution shift analysis)

All methods are trained purely from a fixed offline dataset, with no further environment interaction during offline training.

Behavior Cloning: Setup

BC treats offline RL as supervised classification:

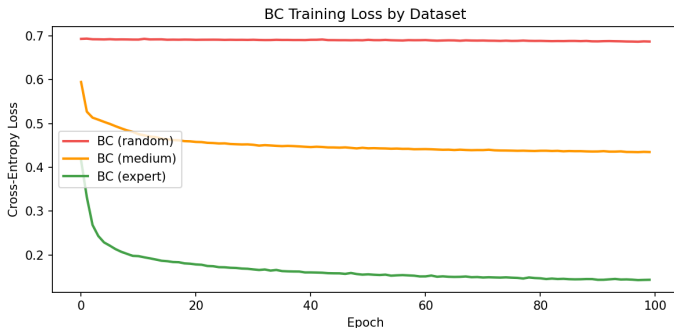
$$\pi_{\theta} = \arg \max_{\pi} \mathbb{E}_{(s,a) \sim \mathcal{D}} [\log \pi(a|s)]$$

Implementation:

- Input: state $s \in \mathbb{R}^4$
- Output: 2 actions
- Loss: CrossEntropyLoss
- No Bellman backup
- No reward signal
- 100 epochs per dataset
- Batch size: 64

Key distinction from Q-learning: BC only sees (s, a) pairs - rewards and next states are completely ignored.

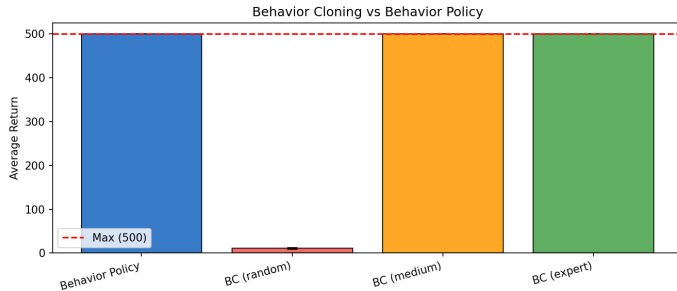
Behavior Cloning: Training Loss



Dataset	Final Loss	Interpretation
Random	≈ 0.693	Stuck at $\log 2$ - entropy of uniform distribution
Medium	≈ 0.44	Partial structure learned
Expert	≈ 0.15	Strong consistent pattern learned

BC (random) converges to $\log 2 \approx 0.693$, the cross-entropy of a uniform binary classifier. This means the network learned nothing useful: the random dataset has no consistent action pattern to imitate.

Behavior Cloning: Evaluation Results



Method	Avg Return	Observation
Behavior Policy	500	Upper bound for BC
BC (random)	~10	Complete failure
BC (medium)	~500	Matched behavior policy
BC (expert)	~500	Matched behavior policy

BC (medium) reaching 500 is explained by the bimodal dataset structure - the 85 episodes scoring 500 in the medium dataset dominate the learned policy. BC learned from the high-quality subset despite the noisy data.

Behavior Cloning: Distribution Shift Analysis



Behavior Cloning: Distribution Shift — Interpretation

Top row — BC (random): large mismatch across all 4 state features.

- Random dataset covers a broad but structureless region
- Deployed BC policy visits very different states
- Result: policy operates out-of-distribution immediately

Bottom row — BC (expert): dataset and deployed states nearly identical.

- Expert dataset covers a narrow, consistent region
- Deployed policy stays within training distribution
- Result: near-zero distribution shift

Theorem 2.1 (Levine et al. 2020):

$$\ell(\pi) \leq C + H^2\epsilon$$

Distribution shift compounds *quadratically* with horizon H . BC (random) makes small errors early, enters unseen states, and keeps compounding explaining its near-zero return.

BC Summary: What We Learned

Finding	Explanation
BC (random) fails completely	No consistent pattern to imitate. Loss stuck at $\log 2$.
BC (medium) matches expert	Bimodal dataset — BC learned from the high-return subset.
BC (expert) matches behavior policy	Low distribution shift. Expert data is consistent and near-optimal.
BC cannot improve beyond π_β	Hard ceiling: $\pi_{BC} \approx \pi_\beta$.

This motivates offline Q-learning and CQL:

Can we use the reward signal to improve *beyond* imitation?